

```

// EMERGENCE OS 1 - Output Modifier
const modifier = (text) => {
  try {
    EmergenceEngine.init();

    if (state.emergence && state.emergence.commandOutput) {
      const cmdBox = state.emergence.commandOutput;
      state.emergence.commandOutput = null;
      return { text: cmdBox };
    }

    if (!text) return { text: "\u200B" };

    // Pull the hidden profile-fill and reflection tags out first, so
    // keyword/NPC detection below only ever sees what the player actually sees.
    const profileResult = EmergenceEngine.extractAndStripTag(text, "EOS_PROFILE")
    let visibleText = profileResult.text;
    if (profileResult.block) {
      EmergenceEngine.applyProfileFill(EmergenceEngine.parseTagFields(profileResu
    }

    const reflectionResult = EmergenceEngine.extractAndStripTag(visibleText, "EOS
    visibleText = reflectionResult.text;
    if (reflectionResult.block) {
      const data = EmergenceEngine.parseTagFields(reflectionResult.block);
      if (data.name && data.text) EmergenceEngine.appendReflection(data.name, dat
    }

    const lower = visibleText.toLowerCase();
    let modifiedText = visibleText;

    if (!state.emergence.initialized && state.emergence.turnCount <= 1) {
      state.emergence.initialized = true;
      modifiedText += `
      [EMERGENCE OS 1: Active - Architecture Online. Type
    }

    // Locations first: a specific "in/at/entering X" match is a more confident
    // signal than the bare-capitalized-word character heuristic, so letting
    // locations claim a name first stops places being misread as NPCs.
    EmergenceEngine.discoverLocations(visibleText);
    EmergenceEngine.updateLocationConditions(visibleText);
    if (state.emergence.config.HumanAgency === "Enabled") {
      EmergenceEngine.discoverCharacters(visibleText);
    }

```

```

if (state.emergence.config.WorldTensionEngine === "Dynamic") {
  if (EmergenceEngine.getTriggerPatterns().tensionUp.test(lower)) {
    EmergenceEngine.updateStat(state.emergence, 'worldTension', 15);
  } else if (state.emergence.worldTension > 10) {
    EmergenceEngine.updateStat(state.emergence, 'worldTension', -2);
  }
}

if (state.emergence.config.PlayerTrauma === "Enabled") {
  if (EmergenceEngine.getTriggerPatterns().traumaUp.test(lower)) {
    EmergenceEngine.updateStat(state.playerInner, 'stress', 12);
    EmergenceEngine.updateStat(state.playerInner, 'egoReserve', -10);
  } else if (EmergenceEngine.getTriggerPatterns().traumaDown.test(lower)) {
    EmergenceEngine.updateStat(state.playerInner, 'stress', -10);
    EmergenceEngine.updateStat(state.playerInner, 'egoReserve', 15);
  }

  if (state.playerInner.stress > 75) state.playerInner.condition = "Traumatiz
else if (state.playerInner.stress > 50) state.playerInner.condition = "On E
else if (state.playerInner.egoReserve < 35) state.playerInner.condition = "
else state.playerInner.condition = "Calm";
}

if (state.world && state.world.npcs && state.emergence.config.NPCBrainSystem
const activeNames = Object.keys(state.world.npcs);
const presentThisTurn = activeNames.filter(n => new RegExp(`\\b${EmergenceE
const newCrisisNames = [];
activeNames.forEach(name => {
  const npc = state.world.npcs[name];
  if (new RegExp(`\\b${EmergenceEngine.escapeRegExp(name)}\\b`, 'i').test(v
    // Grounds the NPC in a real place and time, not just "always available
    // lets /npc show where and when they were last actually present.
    npc.lastSeenLocation = state.emergence.currentLocation;
    npc.lastSeenTurn = state.emergence.turnCount;
    EmergenceEngine.updateStat(npc, 'egoReserve', -3);

    // An NPC caught up in real danger reacts personally, not just when
    // the player confronts them directly - an explosion in the room
    // should visibly rattle whoever's standing in it.
    if (state.emergence.config.WorldTensionEngine === "Dynamic" && Emergenc
      EmergenceEngine.updateStat(npc, 'stress', 15);
  }

  if (EmergenceEngine.getTriggerPatterns().pushback.test(lower)) {
    const autonomy = EmergenceEngine.contextualPushbackMultiplier(npc);
    if (state.emergence.config.GrudgeTracking === "Enabled") {

```

```

        EmergenceEngine.updateStat(npc, 'grudge', Math.round(20 * autonomy))
        EmergenceEngine.updateStat(npc, 'trust', Math.round(-15 * autonomy))
    }
    EmergenceEngine.updateStat(npc, 'stress', Math.round(20 * autonomy));
    npc.outerMask = "Hostile, pushing back against unearned demands.";
    npc.innerMind = "Furious; looking for an opening or planning to walk
    npc.memories.push(`Player forced them: "${EmergenceEngine.excerptFrom
    if (npc.memories.length > 3) npc.memories.shift();
} else if (EmergenceEngine.getTriggerPatterns().respect.test(lower)) {
    const trustGain = EmergenceEngine.trustGainMultiplier(npc.cognitiveBi
    EmergenceEngine.updateStat(npc, 'trust', Math.round(10 * trustGain));
    EmergenceEngine.updateStat(npc, 'grudge', -5);
    npc.outerMask = "Receptive but cautious.";
    npc.memories.push(`Player showed respect: "${EmergenceEngine.excerptF
    if (npc.memories.length > 3) npc.memories.shift();
}

if (EmergenceEngine.getTriggerPatterns().betrayal.test(lower)) {
    const severity = EmergenceEngine.severityMultiplier();
    const grudgeBefore = npc.grudge;
    EmergenceEngine.updateStat(npc, 'grudge', Math.round(30 * severity));
    EmergenceEngine.updateStat(npc, 'trust', Math.round(-30 * severity));
    npc.innerMind = "Disgusted by the betrayal. Will not forget this.";
    npc.memories.push(`Caught Player lying/betraying: "${EmergenceEngine.
    if (npc.memories.length > 3) npc.memories.shift();
    // A betrayal severe enough to be genuinely defining gets remembered
    // permanently, separate from the rolling memory buffer that
    // otherwise cycles out after a few turns.
    if (grudgeBefore < 70 && npc.grudge >= 70) {
        EmergenceEngine.setFormativeMemory(npc,
            `The moment ${name} stopped trusting the player - a betrayal they
            `This is where ${name} stopped trusting the player.`);
    }
}

if (state.emergence.config.RomanceEngine !== "Disabled" && EmergenceEng
    const pacing = EmergenceEngine.romancePacingMultiplier();
    // Romance doesn't land the same way on someone who's currently
    // furious at you - a real grudge dampens how fast attraction grows.
    const grudgeDamp = npc.grudge > 50 ? 0.4 : (npc.grudge > 25 ? 0.75 :
    const stageBefore = EmergenceEngine.getRelationshipStage(npc.attracti
    EmergenceEngine.updateStat(npc, 'attraction', Math.round(20 * pacing
    const stageAfter = EmergenceEngine.getRelationshipStage(npc.attractio
    if (stageAfter !== stageBefore) {
        state.emergence.pendingRomanceStageChange.push({ name: name, stage:
        if (stageAfter === "In Love") {
            EmergenceEngine.setFormativeMemory(npc,

```

```

        `The moment ${name} realized they had fallen for the player com
        `This is where ${name} fell for the player completely.`);
    }
}

if (state.emergence.config.JealousyMechanic === "Enabled") {
    activeNames.forEach(otherName => {
        if (otherName !== name && state.world.npcs[otherName].attraction
            EmergenceEngine.updateStat(state.world.npcs[otherName], 'grudge
            state.world.npcs[otherName].innerMind = EmergenceEngine.jealous
            state.world.npcs[otherName].memories.push(`Saw Player intimatel
            if (state.world.npcs[otherName].memories.length > 3) state.worl
            EmergenceEngine.nudgeUndercurrent(otherName, name, -20);
        }
    });
}

// EARNED SECURITY: real attachment research (Mikulincer & Shaver)
// documents that sustained secure treatment in a relationship can
// shift someone's attachment pattern over time, not just their mood.
// Modeled here as a streak of genuinely secure conditions holding
// across many separate moments with this NPC, not a single good
// scene - and it's one-directional and permanent, same as the
// research frames it: earned, not given, and not lost by one bad turn.
if (npc.attachmentStyle !== "Secure") {
    if (npc.trust > 75 && npc.grudge < 15) {
        npc.secureStreak = (npc.secureStreak || 0) + 1;
        if (npc.secureStreak >= 10) {
            npc.attachmentStyle = "Secure";
            EmergenceEngine.setFormativeMemory(npc,
                `The moment ${name} realized they could finally trust someone c
                `This is where ${name} learned to trust again.`);
            state.emergence.pendingSecurityShift.push(name);
        }
    } else {
        npc.secureStreak = 0;
    }
}

const threatBefore = npc.threatState;
npc.threatState = EmergenceEngine.getThreatState(npc.stress, npc.cognit
const wasCrisis = threatBefore === "Freeze" || threatBefore === "Fight"
const isCrisis = npc.threatState === "Freeze" || npc.threatState === "F
if (isCrisis && !wasCrisis) {
    EmergenceEngine.setFormativeMemory(npc,
        `The moment ${name}'s composure broke entirely in the player's pres
        `This is where ${name}'s composure broke entirely.`);
}

```

```

        newCrisisNames.push(name);
    }
    if (npc.memories.length > 3) npc.memories.shift();
}
});

// Passive NPC-to-NPC dynamics: reads what already happened in the text,
// no scheduling, no write-back request.
EmergenceEngine.updateUndercurrents(visibleText);

// EMOTIONAL CONTAGION: a real, documented phenomenon (Hatfield, Cacioppo
// & Rapson) - someone else's panic visibly rattles people who witness it,
// not just the person having the crisis. Small and bounded (a flat
// stress bump, once per witness, only from NPCs who just NOW broke) so
// it can't compound into a runaway cascade turn after turn.
if (newCrisisNames.length && state.emergence.config.WorldTensionEngine ===
    newCrisisNames.forEach(sourceName => {
        presentThisTurn.forEach(otherName => {
            if (otherName === sourceName) return;
            EmergenceEngine.updateStat(state.world.npcs[otherName], 'stress', 8);
        });
    });
}
}

// Never return a blank/whitespace-only turn: AI Dungeon treats that as an
// empty response and shows a "please select continue" break in immersion.
if (!modifiedText || !modifiedText.trim()) {
    modifiedText = "\u200B";
}

return { text: modifiedText };
} catch (error) {
    return { text: text || "\u200B" };
}
};
modifier(text);

```